



Planning over actions: Agentic reasoning for semi-structured table question answering

Jianghong Huang^{ID}, Shangbo Yun^{ID}, Silin Chen^{ID}, Xiaodong Gu^{ID}, Beijun Shen^{ID} *

School of Computer Science, Shanghai Jiao Tong University, 800 Dongchuan Road, Shanghai, 200244, China

ARTICLE INFO

Dataset link: [artifact-PoA \(Original data\)](#)

Keywords:

Semi-structured table question answering
Agentic reasoning
Executable semantic actions
Trajectory-level deliberation

ABSTRACT

Answering questions over semi-structured tables is fundamental to understanding real-world tabular documents, yet remains highly challenging. Irregular layouts — such as multi-level headers, merged cells, and heterogeneous regions — often induce semantic misalignment between questions and table contents. The task also requires multi-step compositional reasoning with cross-column dependencies, conditional logic, and numerical operations, where early errors can propagate. These difficulties are further amplified by domain-specific terminology and implicit conventions common in real-world tables.

To address these challenges, we propose TQA-AGENT, an agentic question answering framework for semi-structured tables. TQA-AGENT formulates Table QA as an agentic search problem over an explicit action space of table actions. The framework involves LLM agents that dynamically explore, evaluate, and refine multiple candidate reasoning paths under complex table structures. An action set over tables is designed, which decomposes complex reasoning processes into interpretable and composable primitives with explicit intermediate states. TQA-AGENT further incorporates a multi-agent collaboration mechanism, in which perception, planning, execution, evaluation, deliberation, and decision agents jointly generate, compare, and select reasoning paths through a trajectory-level deliberation process. This agentic reasoning strategy alleviates path dependency and effectively reduces error propagation in long reasoning chains, leading to more stable and reliable answers.

We conduct extensive experiments on four benchmark datasets — CRT, TReB, SciTab, and SSTQA — covering diverse domains and varying table structures. Across datasets and backbone models, TQA-AGENT consistently outperforms state-of-the-art LLM-based baselines. Using DeepSeek-V3, it improves average accuracy by 13.62, 11.71, 9.34, and 4.39 percentage points over Direct-Prompting, TableLLM, MACT, and ST-Raptor, respectively, and reaches 73.20% accuracy on the structurally complex TReB benchmark. Comparable gains are observed with GPT-4o, demonstrating robustness across backbone models. Further ablation studies reveal that the effectiveness of TQA-AGENT comes from the coordinated interaction among diverse action categories, while the trajectory-level deliberation mechanism plays a critical role in enhancing reasoning robustness and overall answer quality.

1. Introduction

Semi-structured tables are ubiquitous in real-world information sources, including web pages, spreadsheets, and PDF reports (Gupta et al., 2020; Herzig et al., 2020; Oguz et al., 2022). They carry rich and heterogeneous information in various domains,

* Corresponding author.

E-mail addresses: jh.huang@sjtu.edu.cn (J. Huang), sjtu_yun@sjtu.edu.cn (S. Yun), csl2457029646@163.com (S. Chen), xiaodong.gu@sjtu.edu.cn (X. Gu), bjshen@sjtu.edu.cn (B. Shen).

<https://doi.org/10.1016/j.ipm.2026.105092>

Received 25 February 2026; Received in revised form 5 August 2026; Accepted 6 August 2026

Available online 14 August 2026

0306-4573/© 2026 Published by Elsevier Ltd.

Cost Item	Planned Number			Actual Number			Completion Percentage	
	Output	Cost		Output	Cost		Output	Cost
		Amount	Percentage		Amount	Percentage		
Variable Cost	500	235000	0.80	600	275200	0.85	1.20	1.17
Fixed Cost		59000	0.20		48000	0.15		0.81
Total		294000	1		323200	1		1.10

 **Question:** What is the total amount of the planned unit cost and the actual unit cost?

 **MACT:** 569200 

 **TQA-Agent:** 617200 

Fig. 1. A motivating example of semi-structured Table QA from the SSTQA dataset. The figure illustrates (top) a product cost summary table with hierarchical headers and merged cells, (middle) a natural-language query asking for the total of planned and actual unit costs, and (bottom) the answers predicted by MACT and TQA-AGENT, where only TQA-AGENT produces the correct result.

such as business analytics, financial reporting, and scientific communication, serving as important resources for information access, analysis, and decision-making. Semi-structured table question answering (Table QA) (Herzig et al., 2020; Pasupat & Liang, 2015; Yin et al., 2020) aims to answer natural language questions by reasoning over tabular content. By bridging natural language understanding and structured data reasoning, Table QA provides an effective means for models to interpret, analyze, and extract insights from complex semi-structured tables.

Despite its practical significance, semi-structured Table QA remains a challenging problem due to several fundamental characteristics of real-world tables.

Challenge 1: Complex and irregular table layouts. Unlike well-structured relational tables, semi-structured tables commonly exhibit complex and irregular layouts, including multi-level headers, merged cells, non-rectangular regions, and implicit hierarchical organizations, often accompanied by missing or omitted fields. Consequently, table semantics are no longer localized within individual cells but are distributed across structural constructs such as header hierarchies, merge relations, and row-column alignments, giving rise to a strong structure-as-semantics property (Chen et al., 2021; X. Liu et al., 2022; Yin et al., 2020; Zhang et al., 2020). As illustrated in Fig. 1, correct interpretation requires recovering complete header paths (e.g., “Planned Number” vs. “Actual Number”) by considering hierarchical header structures and cell contexts; merged entries in the “Cost Item” column implicitly group multiple rows under shared categories; and the “Total” row aggregates values across both planned and actual columns. Moreover, merged cells create irregular regions that blur explicit row-column boundaries, and simultaneously encode multiple semantic roles. Such structural complexity substantially complicates reliable parsing and semantic grounding, as the meaning of even a single numerical value depends on its hierarchical context, aggregation scope, and alignment with other table regions. This interdependence hinders accurate information localization and exacerbates the challenge of aligning natural-language questions with relevant table structures (Herzig et al., 2020; X. Liu et al., 2022; Pasupat & Liang, 2015; Yin et al., 2020).

Challenge 2: Multi-step and compositional reasoning. Beyond structural complexity, answering questions over semi-structured tables typically requires multi-step reasoning that interleaves operations such as conditional filtering, cross-row and cross-column references, aggregation, comparison, and numerical computation (Chen et al., 2020; Zhu et al., 2021). These operations are governed by implicit dependency and precedence constraints, which substantially enlarge the reasoning search space. A central difficulty lies in strong path dependency: errors introduced in early reasoning steps can propagate and amplify through subsequent steps, ultimately yielding incorrect answers even when later computations are locally valid. As shown in Fig. 1, multiple plausible reasoning paths may exist for the same question, differing in how table structures are grounded and how intermediate constraints are applied. However, certain existing approaches, such as MACT, typically commit to a single linear reasoning path; once an early misalignment occurs, the error cannot be revised and is propagated to the final result (Bansal et al., 2025; Tyagi et al., 2025). In contrast, TQA-AGENT explicitly explores multiple alternative reasoning paths for the same question, enabling erroneous paths to be identified and discarded while preserving valid ones, as further analyzed in the case study presented in Section 5.

Challenge 3: Domain-specific semantics and knowledge. Semi-structured tables frequently originate from specialized domains such as medicine, finance, law, and engineering, where headers and cell entries encode domain-specific terminology, abbreviations, and implicit conventions (Agarwal et al., 2025). In such settings, accurate interpretation typically requires background knowledge and contextual inference beyond surface-level lexical matching, a difficulty that is further mirrored in domain-dependent natural-language queries (Chen et al., 2020). For instance, financial tables in the CRT and TreB datasets frequently include indicators such as “EPS” or “CAGR” without explicit definitions; answering queries about profitability or growth depends on understanding the financial meaning encoded by these symbols, since they cannot be reliably interpreted as standalone numeric fields (Agarwal et al., 2025). Lacking such semantic grounding, models may produce locally plausible yet conceptually incorrect interpretations, which subsequently propagate into downstream reasoning errors. Prior work has identified terminological ambiguity and insufficient domain knowledge as major sources of failure in complex table reasoning tasks (Zhu et al., 2021).

To address these challenges, recent studies have explored large language model (LLM)-based approaches for semi-structured Table QA (Jiang et al., 2025; Lu et al., 2025). A widely adopted paradigm linearizes tables into textual sequences and directly prompts LLMs to generate answers (Zhang et al., 2025; T. Zhang et al., 2024). More advanced methods introduce explicit

reasoning processes and tool use, decomposing complex queries into multiple steps and interleaving natural-language reasoning with executable table operations to improve transparency and expressiveness (Sholehrasa et al., 2024; Tang et al., 2025; Y. Zhang et al., 2025). Despite their empirical success, these approaches exhibit several inherent limitations. Most notably, they typically rely on single-path, heuristic-driven reasoning, committing early to a specific interpretation of table structure or a particular reasoning path. Given the large and ambiguous reasoning space induced by complex layouts and multi-step operations, early misalignments in structure interpretation or operation scope selection can irreversibly propagate, with limited mechanisms for systematic revision or comparison of alternative hypotheses. Moreover, intermediate reasoning states are often weakly evaluated, and competing reasoning strategies are rarely reconciled, reducing robustness when multiple plausible interpretations of a semi-structured table coexist.

To overcome these limitations, we propose TQA-AGENT, an agentic reasoning framework for semi-structured Table QA. Rather than following a single linear reasoning path, TQA-AGENT reformulates Table QA as structured state-action reasoning over explicit table states and executable table operations. A UCB-guided tree-search framework performs search directly over structured table states, enabling iterative exploration, evaluation, and backtracking of alternative structural hypotheses. To further improve reasoning reliability, we introduce a trajectory-level deliberation mechanism that compares complete executable reasoning trajectories, enabling grounding-aware verification before selecting the final solution.

We evaluate TQA-AGENT on four semi-structured Table QA benchmarks — CRT, TReB, SciTab, and SSTQA — using DeepSeek-V3, GPT-4o, and Qwen2.5-7B-Instruct as backbone models. Experimental results demonstrate that TQA-AGENT consistently outperforms all baseline methods across datasets and backbone models. Specifically, TQA-AGENT achieves average answer accuracies of 71.56% with DeepSeek-V3 and 73.38% with GPT-4o, corresponding to absolute improvements of 4.39%–13.62% and 3.97%–13.10% over strong LLM-based baselines, respectively. With Qwen2.5-7B-Instruct, TQA-AGENT achieves 60.06% average accuracy and outperforms the budget-matched Self-Consistency baseline by 3.07 percentage points. Performance gains remain consistent across different domains, including financial, scientific, and general-purpose tables, and across varying levels of structural complexity, suggesting that the improvements are primarily attributed to the proposed structured search and agentic reasoning mechanisms, while stronger backbone models provide complementary benefits.

In summary, our contributions are threefold:

- We formulate semi-structured Table QA as structured state-action reasoning over explicit table states and executable table operations, enabling interpretable and executable inference.
- We introduce a tree-search-guided reasoning framework that explores structured table states through composable semantic actions tailored for semi-structured tables.
- We propose a trajectory-level deliberation mechanism that performs grounding-aware verification by comparing complete executable reasoning trajectories, leading to more reliable reasoning under complex table structures.

2. Related work

Early studies on *semi-structured Table QA* primarily relied on manually designed rules and templates for question parsing and answer retrieval (Pasupat & Liang, 2015). While such symbolic approaches offer strong interpretability, they suffer from limited adaptability and poor scalability across diverse table structures and query patterns.

These limitations were partially alleviated by neural models that enabled joint representation learning over questions and tables via end-to-end training (Pan et al., 2021), reducing dependence on handcrafted rules. More recently, pre-trained language models (PLMs) have further advanced table understanding through joint pre-training on large-scale tabular and textual corpora, including TaBERT (Yin et al., 2020), TAPAS (Herzig et al., 2020), TAPEX (Liu et al., 2022), and OmniTab (Jiang et al., 2022). As model scale has increased, researchers have begun transferring the capabilities of LLMs to the tabular domain. For example, TableLLaMA (T. Zhang et al., 2024) and TableLLM (Zhang et al., 2025) improve robustness to hierarchical headers, complex cell contents, and heterogeneous table formats by fine-tuning on large-scale table-oriented instruction data.

With the emergence of strong reasoning capabilities in LLMs, recent work has explored reasoning-centric frameworks for complex Table QA (Lu et al., 2025). A representative line of research adopts the ReAct framework (Yao et al., 2022) to decompose Table QA into multi-step reasoning processes with explicit tool invocation. StructGPT (Jiang et al., 2023) iteratively refines intermediate reasoning states through structured interactions, while ReAcTable (Zhang et al., 2024) interleaves natural-language reasoning with executable table operations. MACT (Zhou et al., 2025) further formulates Table QA as an online planning problem with a multi-agent architecture that explicitly decouples planning and execution via external tool verification. Despite their flexibility, these approaches rely on single-path, heuristic-driven reasoning, limiting systematic exploration, comparison, and revision of alternative reasoning paths.

Another line of LLM-based methods improves reasoning reliability by grounding inference in explicit structured representations and executable operations. RePairTQA (Y. Zhang et al., 2025) examines how structural variations in table representations affect QA performance under identical content, while Knowledge-in-Triples (Sholehrasa et al., 2024) grounds LLM reasoning by extracting semantic triples and incorporating retrieval-augmented generation. ST-Raptor (Tang et al., 2025) further enforces step-level executability by converting tables into hierarchical operator trees and decomposing questions into executable operation sequences. While these structure-grounded approaches enhance reliability through explicit constraints, they typically follow a single deterministic reasoning path once the intermediate representation is fixed.

In contrast, TQA-AGENT casts semi-structured Table QA as a structured search over compositional, table-centric actions. It performs tree-search-guided exploration to construct and refine multiple reasoning trajectories, while multi-agent deliberation evaluates competing paths through cross-trajectory analysis and selects the most reliable answer.

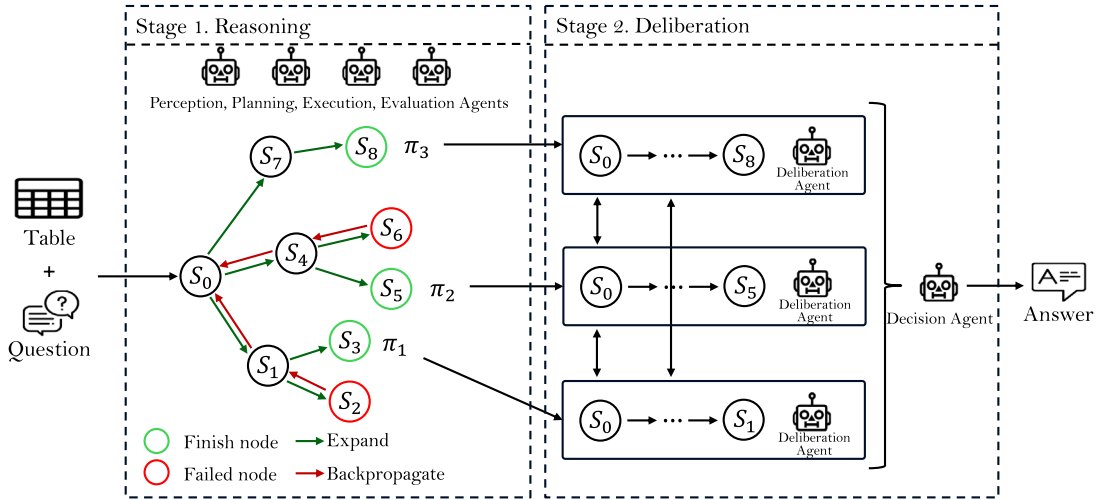


Fig. 2. Overview of TQA-AGENT. In Stage 1 (Reasoning), Perception, Planning, Execution, and Evaluation Agents collaboratively explore the reasoning space from the initial node through a UCB-guided tree search procedure. In Stage 2 (Trajectory-level Deliberation), multiple Deliberation Agents present, challenge, and compare the candidate reasoning paths, after which the Decision Agent selects the optimal path to produce the final answer.

3. TQA-AGENT

3.1. Overview

We study the task of semi-structured Table QA, whose objective is to generate a faithful and semantically correct answer Y given a semi-structured table T and a natural-language question Q . Faithfulness requires that the answer be grounded in the table content and structure, while semantic correctness requires that all constraints expressed in the question be satisfied.

Formally, the task can be defined as learning a mapping function:

$$Y = f(Q, T). \quad (1)$$

To address this problem, TQA-AGENT formulates semi-structured Table QA as a structured search problem over an explicit and executable space of semantic actions, as shown in Fig. 2. In contrast to ReAct-style linear reasoning, which commits early to a single reasoning path, TQA-AGENT maintains a tree-structured reasoning space during inference through tree-search-guided exploration. This formulation allows multiple candidate reasoning strategies to be explored in parallel and compared systematically, with each root-to-leaf path representing a complete reasoning trajectory composed of table-centric actions.

During the reasoning stage, TQA-AGENT coordinates a set of specialized agents within a tree-search-guided framework to collaboratively explore and evaluate reasoning paths over semi-structured tables. The search proceeds over a shared reasoning space, where each node represents the current progress of table transformation, question decomposition, and accumulated intermediate evidence. As the reasoning tree is expanded, TQA-AGENT incrementally constructs a diverse set of candidate reasoning paths that differ in structural grounding decisions, operation ordering, and intermediate constraint resolution. At each node, agents interact in a tightly coupled perception–planning–execution–evaluation loop. The Perception Agent abstracts the current reasoning status into a compact context representation; the Planning Agent selects the next executable action from a predefined action space; the Execution Agent applies this action to produce explicit and inspectable intermediate artifacts; and the Evaluation Agent assesses the resulting state and propagates feedback to guide subsequent exploration. This iterative process continues until terminal nodes are reached, yielding a set of completed candidate reasoning paths.

Beyond candidate generation, TQA-AGENT introduces a second-stage trajectory-level deliberation mechanism to further enhance robustness and reliability. After generating multiple completed reasoning trajectories through tree-search-guided reasoning, Deliberation Agents perform trajectory-level verification via structured comparison and cross-trajectory evaluation. A Decision Agent then aggregates these assessments and selects the most reliable reasoning path as the final solution. This two-stage design ensures that the final answer benefits both from systematic multi-path search and from collective evaluation across competing reasoning strategies.

As summarized in Algorithm 1, the overall framework consists of two key stages: (1) a tree-search-guided reasoning stage that constructs diverse candidate reasoning paths through agent-driven state evaluation (Lines 4–25), and (2) a trajectory-level deliberation stage that evaluates and selects the optimal path among them (Lines 27–29). The inner continuation loop in Algorithm 1 implements a local branch-continuation pruning mechanism rather than a greedy commitment strategy. Specifically, when an expanded child node fails to improve upon its parent, the current branch extension is terminated to prevent unnecessary trajectory growth. However, the child node is retained in the search tree, and its value contribution is still propagated through back-propagation

to update ancestor statistics. In subsequent iterations, the UCB-based selection procedure restarts from the root, allowing alternative branches and previously under-explored nodes to remain eligible for exploration and expansion. Therefore, TQA-AGENT preserves multi-path search behavior across iterations, and the final answer is selected only after multiple completed trajectories are compared through trajectory-level deliberation.

The following sections describe the design of the executable action space (Section 3.2), the tree-search-guided reasoning framework (Section 3.3), the instantiation of specialized agents within this framework (Section 3.4), and the trajectory-level deliberation mechanism for final decision making (Section 3.5).

Algorithm 1: TQA-AGENT Algorithm.

Input: Semi-structured table T , question Q , maximum search iterations N , maximum number of completed trajectories K
Output: Final answer Y

```

1   $\mathcal{T}.root \leftarrow (T, Q, \emptyset)$ ; // Initialize Search Tree  $\mathcal{T}$ 
2   $\mathcal{F} \leftarrow \emptyset$ ; // Set of Finish nodes
3   $iter \leftarrow 0$ ;
4  while  $iter < N$  and  $|\mathcal{F}| < K$  do
5       $iter \leftarrow iter + 1$ ;
6       $currentNode \leftarrow \text{Select}(\mathcal{T}.root)$ ; // Tree Selection
7       $continueRollout \leftarrow \text{true}$ ;
8      while  $continueRollout$  do
9           $parentValue \leftarrow currentNode.value$ ;
10          $childNode \leftarrow \text{Expand}(currentNode)$ ; // Tree Expansion
11          $report \leftarrow \text{Perceive}(childNode.T, childNode.Q, childNode.C)$ ; // Perception Agent
12          $action \leftarrow \text{Plan}(report)$ ; // Planning Agent
13          $childNode.C.action \leftarrow action$ ;
14          $\text{Execute}(action)$ ; // Execution Agent
15          $childNode.value \leftarrow \text{Evaluate}(childNode)$ ; // Evaluation Agent
16          $\text{Backpropagate}(childNode)$ ; // Tree Back-propagation
17         if  $action.type = \text{Finish}$  then
18              $\mathcal{F} \leftarrow \mathcal{F} \cup \{childNode\}$ ;
19              $continueRollout \leftarrow \text{false}$ ;
20         end
21         else if  $childNode.value \leq parentValue$  then
22              $continueRollout \leftarrow \text{false}$ ;
23         end
24         else
25              $currentNode \leftarrow childNode$ ;
26         end
27     end
28 end
29  $\mathcal{P} \leftarrow \text{ExtractTrajectories}(\mathcal{F})$ ;
30  $reports \leftarrow \text{Deliberate}(\mathcal{P})$ ; // Deliberation Agent
31  $\pi^* \leftarrow \text{Determine}(reports)$ ; // Decision Agent
32  $Y \leftarrow \text{Extract}(\pi^*, T, Q)$ ;
33 return  $Y$ ;

```

3.2. Action space design

To address the structural and semantic challenges inherent in semi-structured tables — including multi-level headers, merged cells, cross-region dependencies, domain-specific knowledge, and complex question formulations — we define thirteen executable actions organized into six functional categories: table information retrieval, reasoning, computation, external knowledge retrieval, question decomposition, and termination. The action space provides a structured and executable interface for the reasoning patterns commonly encountered in semi-structured Table QA, allowing tree search to explore over well-defined state transitions instead of unconstrained natural-language reasoning. It is designed to cover representative reasoning operations commonly encountered in semi-structured Table QA. Formally, the action space is defined as

$$\mathcal{A} = \{\mathcal{A}_1, \mathcal{A}_2, \dots, \mathcal{A}_{13}\}. \quad (2)$$

Collectively, these actions form the semantic foundation of the reasoning search space, enabling explicit and interpretable multi-step inference.

Table information retrieval actions. Actions $\{\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3\}$ explicitly parse table structures and localize question-relevant components.

- $\mathcal{A}_1$ (*Header parsing*): flattens multi-level headers into a unified attribute space by resolving hierarchical relations and completing implicit labels.

- $\mathcal{A}_2$ (*Column localization*): identifies question-relevant columns by aligning question semantics with header attributes.
- $\mathcal{A}_3$ (*Row localization*): filters rows based on constraints derived from the question, such as entity mentions or value conditions.

Reasoning actions. Actions $\{\mathcal{A}_4, \mathcal{A}_5, \mathcal{A}_6, \mathcal{A}_7\}$ construct intermediate semantic structures to support multi-step reasoning.

- $\mathcal{A}_4$ (*Column construction*): creates derived columns to store intermediate results or logical indicators.
- $\mathcal{A}_5$ (*Row construction*): adds derived rows representing aggregated or intermediate computation results.
- $\mathcal{A}_6$ (*Row sorting*): orders rows according to a specified attribute for subsequent comparison-based reasoning.
- $\mathcal{A}_7$ (*Grouping*): clusters rows by shared attribute values to support group-wise analysis.

Computation action. Action $\mathcal{A}_8$ performs arithmetic or logical calculations on the table's data.

- $\mathcal{A}_8$ (*Computing*): generates and executes arithmetic or logical formulas over selected table cells and writes the results back to the table.

External knowledge retrieval actions. Actions $\{\mathcal{A}_9, \mathcal{A}_{10}\}$ supplement table information with external knowledge when attribute semantics are ambiguous.

- $\mathcal{A}_9$ (*General retrieval*): retrieves explanatory knowledge for general concepts from open-domain sources.
- $\mathcal{A}_{10}$ (*Domain-specific retrieval*): retrieves professional knowledge from domain-specific knowledge bases to interpret specialized indicators.

Question decomposition actions. Actions $\{\mathcal{A}_{11}, \mathcal{A}_{12}\}$ explicitly model multi-step reasoning by decomposing complex questions.

- $\mathcal{A}_{11}$ (*Parallel decomposition*): splits a complex query into multiple independent sub-questions.
- $\mathcal{A}_{12}$ (*Serial decomposition*): decomposes a query into a sequence of interdependent sub-questions.

Termination action. Action $\mathcal{A}_{13}$ terminates a reasoning path and produces a candidate answer.

- $\mathcal{A}_{13}$ (*Finish*): ends the current reasoning path and generates a candidate answer based on the accumulated state.

At each step t , TQA-AGENT selects an action $a_t \in \mathcal{A}$ based on the tree policy and executes it to transition the current state to its successor.

3.3. Tree-search-guided reasoning

3.3.1. State definition

TQA-AGENT employs a UCB-guided tree-search framework to guide dynamic reasoning over semi-structured tables, as shown in Fig. 3. The search is conducted over a tree of reasoning states, where each node represents partial progress in table transformation and question resolution. This design allows multiple reasoning trajectories to be explored, with the UCB policy allocating more search effort to promising states. We use s_t to denote the reasoning state at step t along a path, and use s to denote a generic node in the search tree when the temporal order is not required. Each reasoning state is defined as:

$$s_t = (T_t, Q_t, C_t), \quad (3)$$

where T_t denotes the current table representation after executing a sequence of actions, Q_t denotes the current question representation used for guiding the next action selection, and C_t denotes the accumulated information produced during reasoning, including executed actions and intermediate results.

3.3.2. Selection, expansion, state evaluation, and back-propagation

Following a tree-search-guided framework inspired by Monte Carlo Tree Search (MCTS) (Browne et al., 2012), the reasoning process iteratively executes four steps: *selection*, *expansion*, *state evaluation*, and *back-propagation*. Different from classical MCTS, which typically relies on stochastic rollouts for value estimation, our method adopts structured, agent-driven state evaluations that explicitly model table semantics, intermediate constraints, and logical consistency. This modification transforms tree search from purely statistical exploration into a semantics-aware reasoning process, enabling dynamic adjustment of trajectories based on intermediate outcomes while preserving interpretability.

Selection. Starting from the root, the algorithm recursively traverses the tree to identify a promising non-terminal node for expansion. For each state s_t , we maintain a visit count $N(s_t)$ and a value estimate $Q(s_t)$. Given a parent state s_t , the next state to explore is selected from its children according to the Upper Confidence Bound (UCB) criterion:

$$\text{Select}(s_t) = \arg \max_{s' \in \text{Children}(s_t)} \left(Q(s') + c \sqrt{\frac{\ln N(s_t)}{N(s') + 1}} \right), \quad (4)$$

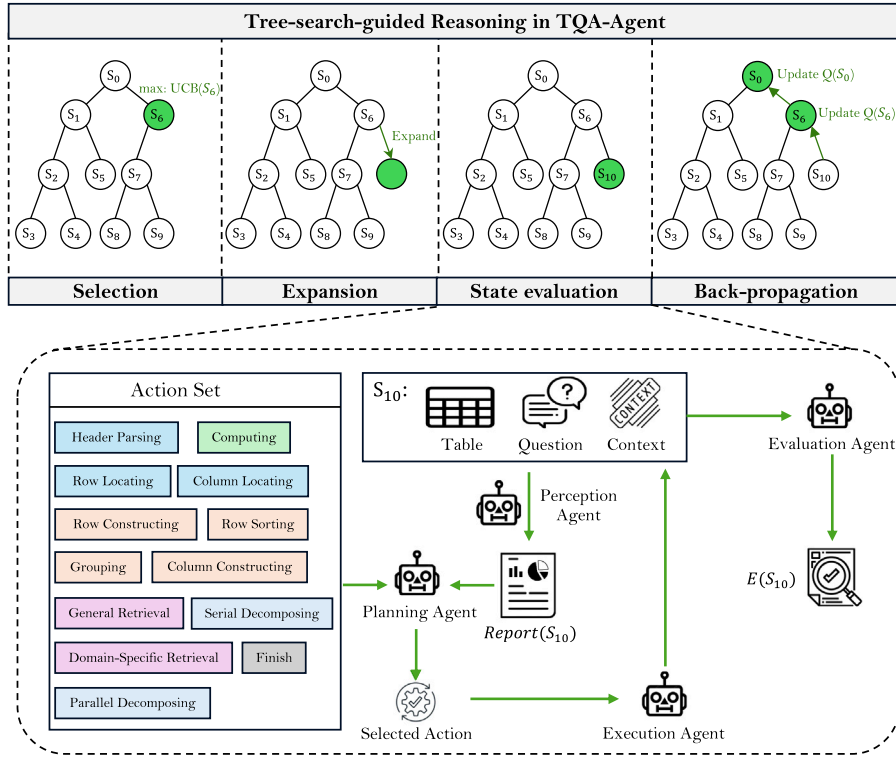


Fig. 3. Tree-search-guided dynamic reasoning. Starting from the root node, TQA-AGENT selects promising nodes using the UCB strategy and expands the selected node. On each newly expanded node, a state-evaluation step is performed, where an action is planned based on the current context, executed on the table, and then evaluated. The evaluation result is back-propagated along the reasoning path to update the statistics and value estimates of the visited nodes.

where c controls the exploration–exploitation trade-off. The term $Q(s')$ reflects the accumulated reasoning quality of child state s' , while the second term encourages exploration of less-visited branches. This mechanism promotes broad exploration in early stages and gradually concentrates computational resources on higher-quality trajectories as evidence accumulates.

Expansion. Once a leaf node is selected, a new child state is instantiated to extend the reasoning tree. Unlike traditional MCTS, which expands nodes by enumerating untried actions, TQA-AGENT separates structural expansion from semantic execution. Specifically, given the current state s_t , a successor state s_{t+1} is created and attached as a new node in the tree, serving as the execution context for the subsequent reasoning step. The actual action decision and semantic evolution are deferred to the state-evaluation step. This separation decouples tree topology growth from action realization, improving structural clarity and interpretability.

State Evaluation. Instead of stochastic rollouts, TQA-AGENT performs deterministic state evaluations within the tree-search-guided framework through coordination among specialized agents. This process provides controllable state transitions and interpretable search dynamics over structured table states. Given a state s_t , the Perception Agent first constructs a structured state representation that captures the transformed table schema, resolved and unresolved constraints, and intermediate reasoning artifacts maintained in memory. Based on this representation, the Planning Agent selects an action $a_t \in \mathcal{A}$ by jointly considering structural feasibility (e.g., schema compatibility and operational executability) and semantic alignment with the target query. The Execution Agent then deterministically applies a_t to generate the successor state s_{t+1} while consistently updating intermediate data structures and memory variables. Finally, the Evaluation Agent assigns a scalar value $E(s)$, which is used for value back-propagation during tree search.

The evaluation function consists of a heuristic structural component and a semantic completion component. For non-terminal states, the heuristic component measures structural understanding, reasoning progress, and answer-generation status through the following weighted formulation:

$$E_{\text{base}}(s) = 0.20 f_{\text{header}}(s) + 0.25 f_{\text{col}}(s) + 0.20 f_{\text{row}}(s) + 0.25 f_{\text{num}}(s) + 0.20 f_{\text{ans}}(s) - 0.15 f_{\text{step}}(s). \quad (5)$$

Here, f_{header} indicates whether header semantics are correctly identified; f_{col} and f_{row} measure the resolution ratios of relevant columns and rows; f_{num} captures the amount of useful numeric intermediate information stored in memory (with an upper bound to prevent uncontrolled accumulation); f_{ans} indicates whether a candidate answer has been generated; and f_{step} penalizes unnecessarily long reasoning trajectories through normalized path length.

For terminal states, we introduce a task-level completion reward:

$$\gamma_{\text{term}}(s) = \begin{cases} 0.6, & \text{if a required numeric answer is successfully produced,} \\ -0.2, & \text{if a required numeric answer is missing,} \\ 0.2, & \text{otherwise.} \end{cases} \quad (6)$$

The reward for numeric-answer tasks only checks answer-type compatibility and format validity. Specifically, it verifies whether the terminal state contains a parseable numeric output when such an output is expected. It does not access the gold answer or evaluate whether the predicted value is correct during search. The penalty for missing required numeric outputs discourages premature termination, while the reward for other terminal states provides a completion signal without assuming answer correctness. The coefficients in Eqs. (5) and (6) are empirically selected based on preliminary experiments and further validated through sensitivity analysis. The results show that TQA-AGENT maintains stable performance under moderate variations of these reward configurations.

Beyond heuristic signals, the Evaluation Agent further refines the score based on a structured description of the current node, including the table view, reformulated query, executed action sequence, identified key cells, intermediate results, and detected error signals. The refinement assesses the node along three complementary criteria: (1) *Progress*, measuring the degree to which uncertainty is reduced or reasoning advances toward the target answer; (2) *Consistency*, evaluating structural and semantic coherence between executed table operations and the query intent; and (3) *Validity*, verifying the logical soundness and numerical correctness of intermediate derivations. This component produces an adjustment term $\Delta_{\text{refine}}(s)$ together with a concise explanatory rationale. The resulting score is subsequently normalized to the interval [0, 1].

The overall evaluation is defined as

$$E(s) = \begin{cases} E_{\text{base}}(s) + \Delta_{\text{refine}}(s), & \text{if } s \text{ is non-terminal,} \\ \gamma_{\text{term}}(s) + \Delta_{\text{refine}}(s), & \text{if } s \text{ is terminal.} \end{cases} \quad (7)$$

This hybrid evaluation mechanism integrates interpretable progress-based scoring with semantic-level verification, yielding stable and fine-grained value estimates for guiding tree search.

Back-propagation. After state evaluation, the evaluation signal is propagated backward along the selected path to update the statistics of all ancestor states. Let s denote an intermediate state on the selected path and s' its evaluated successor. The value estimate is updated using an incremental averaging scheme:

$$Q(s) \leftarrow \frac{Q(s) \cdot N(s) + E(s')}{N(s) + 1}, \quad (8)$$

where $N(s)$ denotes the visit count of state s . This update incorporates the newly observed evaluation $E(s')$ into the empirical value estimate while preserving historical information from previous evaluations. By progressively aggregating evidence across multiple search iterations, the value estimates become increasingly reliable, allowing subsequent selection decisions to better prioritize promising reasoning trajectories.

Back-propagation updates only the visit counts and value estimates of the search tree constructed for the current question. Thus, the tree search procedure in TQA-AGENT performs instance-level search refinement and trajectory selection during inference, without updating or optimizing a global reasoning policy across questions.

3.3.3. Termination and candidate path set

A reasoning path is considered complete when an action of type *Finish* is generated, indicating that a candidate answer has been produced. During search, a non-terminal branch extension may also be stopped early when the newly evaluated child does not improve upon its parent; this pruned branch remains in the search tree for statistics but is not added to the candidate path set. The ordered sequence of actions from the root to a *Finish* node defines a complete reasoning path:

$$\pi = a_1 \oplus a_2 \oplus \dots \oplus a_{|\pi|}. \quad (9)$$

The search procedure terminates when either the maximum number of iterations N is reached or the number of completed terminal trajectories reaches a predefined threshold K . Upon termination, all completed reasoning paths collected during search constitute the candidate set:

$$\mathcal{P} = \{\pi_1, \pi_2, \dots, \pi_{|\mathcal{P}|}\}, \quad (10)$$

which is subsequently passed to the trajectory-level deliberation stage for global evaluation and final selection.

3.4. Agent instantiation

To instantiate the tree-search-guided reasoning process, TQA-AGENT employs six specialized agents corresponding to distinct functional roles. The agents interact through a shared reasoning framework, collectively supporting perception, planning, execution, evaluation, deliberation, and decision-making during search.

Perception Agent. The Perception Agent is responsible for constructing a structured abstraction of the current reasoning state prior to action selection. Given a node (T_t, Q_t, C_t) at reasoning step t , it analyzes the current table representation (e.g., expanded headers, filtered rows, or derived attributes), the question representation, and the accumulated reasoning information. Its output is a compact

context report summarizing (1) resolved structural semantics, (2) current reasoning progress toward the target question, and (3) remaining objectives implied by the current state. This explicit representation of reasoning status provides the Planning Agent with a concise state representation that integrates relevant table structures and intermediate reasoning information.

Planning Agent. Based on the context report produced by the Perception Agent, the Planning Agent selects the next executable action from a predefined action set tailored to semi-structured tables (Section 3.2). Its decision considers both local feasibility and global task progression. Actions that are structurally incompatible with the current table state, redundant with previously explored operations, or unlikely to advance unresolved constraints are discouraged. Within the tree-search framework, the Planning Agent determines how nodes are semantically expanded, thereby shaping the topology of the reasoning tree and enabling principled exploration of alternative reasoning strategies.

Execution Agent. The Execution Agent applies the selected action to the current reasoning state. It performs concrete transformations on s_t , such as filtering rows, selecting or aggregating columns, constructing derived attributes, or computing intermediate numerical or logical results. Unlike symbolic planners that operate at an abstract level, the Execution Agent produces explicit and inspectable intermediate artifacts, including sub-tables, aligned row groups, derived columns, and symbolic values. These artifacts are incorporated into the node context and directly influence subsequent perception, evaluation, and back-propagation.

Evaluation Agent. After execution, the Evaluation Agent assigns a deterministic scalar score to the newly generated reasoning state, which is propagated through the search tree to update node value estimates. Instead of relying on stochastic rollouts, the agent evaluates states using interpretable signals directly derived from the current reasoning context. The evaluation is performed in two steps within the agent. First, the agent computes a deterministic base score grounded in the observable reasoning state. For non-terminal states, the score is computed from heuristic features that characterize reasoning progress, including successful parsing of table headers, identification of relevant rows and columns, generation of numeric intermediate variables, and construction of a candidate answer. It also incorporates a normalized path-length penalty used in UCB computation to discourage unnecessarily deep or redundant trajectories. For terminal states, the score is determined by answer-type constraints inferred from the question: if a numeric response is required, the predicted answer is verified for numeric validity; otherwise, a fixed plausibility reward is assigned. Second, the Evaluation Agent refines this base score based on the full node context, including the current table view, the reformulated question, executed actions, key located cells, intermediate results, and any detected error or failure signals. The final score is produced by adjusting the base score according to this refinement and normalizing it to the range [0,1].

In addition to producing quantitative scores, the Evaluation Agent generates qualitative natural-language assessments grounded in the current reasoning state. These qualitative signals provide fine-grained feedback on potential logical inconsistencies, overlooked constraints, or incomplete intermediate steps. Such feedback promotes action diversity during expansion; without it, the search process tends to generate highly similar expansions when revisiting a parent node. By identifying reasoning deficiencies or suggesting alternative considerations, the Evaluation Agent encourages exploration of more diverse and semantically distinct reasoning trajectories, while maintaining stable and low-variance reward signals grounded in the reasoning state.

Deliberation Agent. The Deliberation Agents operate after the reasoning stage has produced a set of complete reasoning trajectories. Each Deliberation Agent is assigned a full path and evaluates it as a coherent reasoning chain. During trajectory-level deliberation, agents articulate the strengths of their assigned trajectories while critically examining alternative paths, focusing on the soundness of intermediate assumptions, evidential sufficiency, and completeness with respect to question constraints. This path-level analysis enables structured comparison beyond local node scores.

Decision Agent. The Decision Agent synthesizes the structured analyses produced by Deliberation Agents and selects the most reliable reasoning path. Its decision is based on global criteria, including completeness, consistency, and evidential support, rather than step-level scores alone. Once the optimal path is selected, the Decision Agent extracts the final answer from the terminal node of that path, ensuring that the output is grounded in a fully traceable reasoning process with explicit intermediate results.

We provide the full set of prompts for all agents in our accompanying artifacts for reference.

3.5. Trajectory-level deliberation and final decision

The tree-search-guided reasoning stage produces a set of candidate reasoning paths, each corresponding to a complete sequence of executable actions that yields a plausible answer. However, due to semantic ambiguity, implicit structural constraints, and competing interpretations in semi-structured tables, selecting a single solution based solely on local search scores may be brittle. To further improve robustness, TQA-AGENT introduces a trajectory-level deliberation mechanism that performs global, path-level evaluation and facilitates principled selection among competing reasoning trajectories. Specifically, the deliberation framework is operationalized through four sequential steps: self-evaluation, trajectory-level deliberation, comprehensive scoring, and final decision-making.

Self-evaluation. Each Deliberation Agent is assigned one candidate reasoning path $\pi \in \mathcal{P}$ and evaluates it as a coherent, end-to-end reasoning path. The agent presents the full action sequence, key intermediate results, any external knowledge utilized, and the resulting conclusion. Based on this holistic assessment, the agent assigns a self-evaluation score $S_{\text{self}}(\pi)$, which reflects the perceived correctness, completeness, and internal consistency of the reasoning path.

Trajectory-level deliberation. After self-evaluation, the retained candidate paths undergo structured pairwise trajectory comparison to evaluate their relative strengths and weaknesses. Rather than assessing each path independently, Deliberation Agents critically examine alternative paths by challenging their underlying assumptions, intermediate transformations, and supporting evidence,

aiming to uncover weaknesses such as invalid constraints, unsupported deductions, incomplete structural alignment, or erroneous table interpretation. For each pairwise comparison, a cross-trajectory consistency score $S_{\text{cross}}(\pi', \pi)$ is computed, measuring the extent to which the challenged path π can address criticisms raised from the perspective of another candidate path $\pi' \in \mathcal{P}$.

Given n retained candidate paths, at most $\binom{n}{2}$ path pairs are evaluated, with each pair compared only once. The resulting pairwise analysis serves as a lightweight verification mechanism for identifying disagreements among competing trajectories, including differences in row selection, header alignment, computation scope, and the interpretation of implicit table semantics, while avoiding the overhead of multi-round exchanges.

Comprehensive scoring. Based on both self-evaluations and cross-trajectory consistency assessments, each reasoning path is assigned a comprehensive score:

$$\text{Score}(\pi) = \lambda_1 \cdot S_{\text{self}}(\pi) + \lambda_2 \cdot \frac{1}{|\mathcal{P}| - 1} \sum_{\pi' \in \mathcal{P}, \pi' \neq \pi} S_{\text{cross}}(\pi', \pi), \quad (11)$$

where λ_1 and λ_2 are non-negative weighting coefficients that sum to one, controlling the balance between internal path quality and cross-trajectory consistency. The averaging operation ensures that the comprehensive score remains invariant to the number of candidate reasoning paths.

Decision-making. All path-level evaluation results are submitted to a dedicated Decision Agent, which conducts the final global arbitration by ranking candidate reasoning paths based on their comprehensive scores, while further accounting for reasoning coverage and logical consistency. The selected path π^* is adopted as the final reasoning path, and its output is returned as the system prediction:

$$Y = \text{Output}(\pi^*). \quad (12)$$

The complete reasoning chain associated with π^* is preserved as an explicit explanation, thereby strengthening both the robustness and the interpretability of the final decision.

4. Experiments

We conduct experiments to evaluate the effectiveness and internal mechanisms of TQA-AGENT, aiming to answer the following research questions:

- **RQ1:** Can TQA-AGENT achieve consistent performance improvements over strong LLM-based baselines across different semi-structured table QA datasets and backbone models?
- **RQ2:** How do different categories of reasoning actions contribute to the overall performance of TQA-AGENT?
- **RQ3:** To what extent do each key component's design choices affect TQA-AGENT's reasoning robustness and answer accuracy?
- **RQ4:** How do hyperparameters such as top- m actions, number of candidate paths, and cross-trajectory consistency scoring weights influence the effectiveness of TQA-AGENT?
- **RQ5:** Can the LLM-based judge provide reliable evaluation results that are consistent with human judgments for semi-structured table QA?

4.1. Experimental setup

4.1.1. Models

We use DeepSeek-V3 (DeepSeek-AI, 2024) and GPT-4o (OpenAI, 2024) as backbone models in the primary experiments. Evaluating TQA-AGENT across these two strong yet architecturally different LLMs demonstrates that its performance improvements are consistent across backbones and arise from the proposed reasoning framework. To further examine its applicability beyond frontier-scale models, we additionally evaluate TQA-AGENT with Qwen2.5-7B-Instruct.

4.1.2. Datasets

We evaluate TQA-AGENT on four representative semi-structured Table QA datasets spanning financial, scientific, and general domains:

- **CRT** (Zhang et al., 2023): This dataset consists of real-world Chinese financial reports, including profit and loss statements, balance sheets, and cash flow statements. Tables in CRT often exhibit complex structures such as multi-level headers, merged cells, and multiple numerical regions. The questions typically involve indicator lookup, period-over-period comparison, and cross-column referencing, making this dataset suitable for evaluating structured reasoning and logical consistency in the financial domain.
- **TReB** (Li et al., 2025): TReB contains a wide range of financial tables with more diverse and less standardized structures, such as enterprise operation summaries and investment analysis reports. Compared with CRT, table layouts in TReB do not necessarily follow unified patterns, and the questions are more open-ended, often requiring conditional filtering, aggregation, or reasoning across different table regions. This dataset is used to evaluate model robustness on non-canonical financial tables.

- **SciTab** (Lu et al., 2023): Comprising tables extracted from scientific research papers, SciTab focuses on experimental settings, performance metrics, and statistical results. These tables typically contain a larger number of attributes with finer-grained row and column organization. The associated questions often require multi-row comparison, attribute selection, or semantic interpretation of specific fields, enabling evaluation of cross-domain structural parsing and reasoning ability.
- **SSTQA** (Tang et al., 2025): This dataset covers a diverse collection of real-world semi-structured tables, including government statistics, product specification sheets, and public reports. The tables exhibit high structural heterogeneity, such as multi-level headers, merged layouts, regional substructures, and missing values. The corresponding questions span entity retrieval, numerical relation judgment, and composite logical reasoning, providing a comprehensive testbed for complex Table QA scenarios.

Together, these datasets exhibit substantial variation in domain semantics, structural regularity, and reasoning complexity. Evaluating TQA-AGENT on all four datasets allows us to assess its effectiveness, robustness, and generalizability across diverse semi-structured table scenarios.

4.1.3. Baselines

We compare TQA-AGENT with several state-of-the-art methods for *semi-structured Table QA*, covering representative paradigms of end-to-end LLM inference, structured prompting, multi-agent reasoning, and explicit table structure modeling. The baselines are:

- **Direct-Prompting**: The table is linearized and concatenated with the question as input to the LLM, which directly generates the answer without explicit structural encoding or intermediate reasoning. This serves as a fundamental reference for assessing the gains of TQA-AGENT over the base model.
- **TableLLM** (Zhang et al., 2025): A structured prompting method that incorporates table layout, attribute labels, and row/column indicators into the input to enhance structural awareness, without modifying model parameters or adding external reasoning modules.
- **MACT** (Zhou et al., 2025): A multi-agent collaborative reasoning framework based on ReAct (Yao et al., 2022), where role-specific agents coordinate and invoke tools for multi-step reasoning while maintaining traceable reasoning trajectories.
- **ST-Raptor** (Tang et al., 2025): A structural modeling approach that maps tables into higher-order table trees (HO-Trees), enabling attribute localization, merged-region parsing, and hierarchical cross-segment reasoning over complex table structures.
- **Self-Consistency**: A budget-matched multi-sampling baseline that independently generates multiple reasoning trajectories for the same table-question input using the same backbone model and selects the final answer via semantic majority voting. The number of sampled trajectories is dynamically calibrated to ensure that its average token consumption remains within approximately 5% of TQA-AGENT, providing a fair comparison under comparable inference budgets.

We discuss StructGPT (Jiang et al., 2023) and ReActTable (Zhang et al., 2024) in the related work section as representative structured-interaction and ReAct-style table reasoning approaches, respectively, but do not include them as primary baselines for the following reasons. StructGPT addresses general structured-data reasoning scenarios, whereas our study focuses on semi-structured tables characterized by merged cells, multi-level headers, and irregular layouts. ReActTable follows a ReAct-style paradigm for table QA, while MACT serves as a stronger and more recent agentic reasoning baseline with explicit planning, execution, and tool verification. Therefore, we select MACT and ST-Raptor as representatives of agentic reasoning and structure-aware semi-structured table modeling, respectively. This choice avoids introducing additional adaptation or reimplementing bias for methods whose original assumptions do not fully align with our task setting.

For fair comparison, we replace the base models in baselines with the same LLMs used in our experiments and align their preprocessing with our datasets.

4.1.4. Evaluation metrics

We evaluate TQA-AGENT using two widely adopted metrics: *Accuracy* and *Avg. Tokens*. *Accuracy* measures whether the model's generated answer is semantically consistent with the ground truth. The assessment is conducted by an LLM-based judge, following common practice in table-based question answering (Nan et al., 2022; Zhou et al., 2024). The evaluation prompt used by the LLM-based judge is publicly released in our accompanying artifact. To mitigate potential evaluation bias, we conduct an independent human study, employ stratified sampling, preserve the original benchmark context, and validate LLM-based judgments against human assessments (Section 4.6).

Avg. Tokens denotes the average number of tokens consumed per question across the entire reasoning pipeline, including planning, execution, evaluation, and trajectory-level deliberation. Since deployment metrics such as inference latency and monetary API cost depend on implementation- and platform-specific factors, we use *Avg. Tokens* as a reproducible and implementation-independent proxy for computational cost, enabling a fair comparison of the accuracy–efficiency trade-off across different methods.

4.1.5. Implementation details

All methods (TQA-AGENT and baselines) are implemented under an identical experimental protocol to ensure fair comparison. Specifically, all approaches use the same backbone LLMs and inference configurations, and experiments are conducted in a zero-shot setting without any task-specific fine-tuning. All backbone models operate in inference-only mode. Unless otherwise specified, the decoding temperature is fixed to 0.0 to ensure deterministic behavior, and random seeds are fixed for reproducibility.

For search-based reasoning, the number of candidate reasoning paths is set to {1, 3, 5, 7}, with five as the default configuration. All reasoning paths are generated under the same action space, stopping criteria, and prompting templates across methods to eliminate

Table 1
Hyperparameters for tree search, planning, evaluation, and selection.

Hyperparameter	Description	Default
<i>Tree search parameters</i>		
exploration_c	UCB exploration constant	1.4
num_iters	Number of tree-search iterations	64
max_candidates	Max candidate reasoning paths retained	5
max_depth	Maximum search depth (heuristic stop)	12
min_score_to_expand	Minimum score threshold to allow expansion	-1e9
seed	Random seed for reproducibility	7
<i>Planning agent parameters</i>		
topk	Candidate actions per state	5
temperature	LLM sampling temperature for planning	0.0
max_tokens	Max tokens for planning LLM call	900
use_context_report	Structured context sensing	True
<i>Evaluation agent parameters</i>		
use_llm	Enable LLM-based evaluation refinement	True
temperature	LLM sampling temperature for evaluation	0.0
max_tokens	Max tokens for evaluation LLM call	500
<i>Selection (Deliberation & Decision) parameters</i>		
temperature	LLM sampling temperature for selection	0.0
max_tokens	Max tokens for selection LLM call	1200
use_verifier	Enable additional verification during decision stage	True
λ_1	Weight for self-evaluation score in comprehensive scoring	0.6
λ_2	Weight for cross-trajectory consistency score in comprehensive scoring	0.4

Table 2
Performance comparison with LLM-based methods on four benchmarks (CRT, TReB, SciTab, and SSTQA) across three backbone models.

Model	Method	Accuracy					Avg. tokens
		CRT	TReB	SciTab	SSTQA	Avg.	
DeepSeek-V3	Direct-Prompting	58.31%	56.60%	53.66%	63.22%	57.94%	1.8k
	TableLLM	59.96%	59.20%	56.09%	64.15%	59.85%	2.2k
	MACT	63.73%	61.80%	58.17%	65.18%	62.22%	4.2k
	ST-Raptor	64.83%	70.60%	60.84%	72.39%	67.17%	5.7k
	Self-Consistency	70.42%	70.60%	60.92%	73.10%	68.76%	23.1k
	TQA-AGENT (Ours)	73.83%	73.20%	63.59%	75.62%	71.56%	23.2k
GPT-4o	Direct-Prompting	61.07%	59.40%	55.32%	65.31%	60.28%	1.9k
	TableLLM	62.99%	61.80%	57.25%	65.97%	62.00%	2.4k
	MACT	66.02%	64.80%	58.41%	67.92%	64.29%	4.5k
	ST-Raptor	68.96%	73.00%	61.50%	74.18%	69.41%	6.3k
	Self-Consistency	72.15%	73.00%	62.75%	74.82%	70.68%	25.3k
	TQA-AGENT (Ours)	75.02%	76.20%	65.27%	77.03%	73.38%	25.5k
Qwen2.5-7B	Direct-Prompting	45.17%	43.60%	41.83%	50.94%	45.39%	1.7k
	TableLLM	47.97%	45.40%	43.13%	52.07%	47.14%	2.1k
	MACT	50.65%	49.20%	45.74%	54.36%	49.99%	3.9k
	ST-Raptor	54.14%	58.20%	48.81%	60.69%	55.46%	5.3k
	Self-Consistency	57.43%	58.80%	49.96%	61.77%	56.99%	21.1k
	TQA-AGENT (Ours)	61.37%	62.00%	52.63%	64.22%	60.06%	21.4k

implementation bias. The detailed hyperparameters for the tree search process, planning agent, evaluation agent, and selection (deliberation and decision) module are summarized in Table 1. Concretely, we set the tree-search exploration constant to 1.4, the number of search iterations to 64, and the maximum search depth to 12 as a heuristic stopping criterion. The final comprehensive score integrates self-evaluation and cross-trajectory consistency signals with weights $\lambda_1 = 0.6$ and $\lambda_2 = 0.4$.

External knowledge augmentation is dataset-specific: FIBO is used for CRT and TReB, ConceptNet-Science for SciTab, and Wikipedia for SSTQA. Knowledge retrieval is performed consistently across all compared methods using the same preprocessing and selection strategy to avoid discrepancies introduced by external resources. In addition, identical table preprocessing results are reused across methods to further ensure comparability.

All experiments are conducted on a server equipped with two NVIDIA GeForce RTX 4090 GPUs (24 GB memory each) under Ubuntu 23.10, with NVIDIA driver 550.54.15 and CUDA 12.4 support. The system is implemented in Python, and LLM inference is conducted via API calls. Full implementation details, prompts, and configuration files are provided in the accompanying artifact to facilitate reproducibility.

Table 3

Stability and significance analysis of TQA-AGENT with DeepSeek-V3. Accuracy is reported as mean $\pm$ sample standard deviation over five runs. The budget-matched Self-Consistency baseline is included for stability comparison, while the statistical tests compare TQA-AGENT with ST-Raptor, the strongest original baseline.

Method	Accuracy				
	CRT	TReB	SciTab	SSTQA	Avg.
ST-Raptor	64.76 $\pm$ 0.64	70.40 $\pm$ 0.72	60.71 $\pm$ 0.58	72.21 $\pm$ 0.61	67.02 $\pm$ 0.42
Self-Consistency	70.42 $\pm$ 1.03	70.60 $\pm$ 0.96	60.92 $\pm$ 0.88	73.10 $\pm$ 0.79	68.76 $\pm$ 0.66
TQA-AGENT	73.83 $\pm$ 0.31	73.20 $\pm$ 0.34	63.54 $\pm$ 0.38	75.58 $\pm$ 0.35	71.54 $\pm$ 0.22
Accuracy gap	+9.07	+2.80	+2.83	+3.37	–
Paired bootstrap p -value	<0.001	0.013	0.017	0.006	–
McNemar p -value	<0.001	0.018	0.021	0.009	–

4.2. Main results (RQ1)

Results are summarized in Table 2. Across all twelve experimental settings — covering four representative datasets and three backbone language models — TQA-AGENT consistently outperforms all baseline methods, achieving the highest accuracy on every dataset. For example, when using DeepSeek-V3 as the base model, TQA-AGENT attains an average accuracy of 71.56%, surpassing Direct-Prompting, TableLLM, MACT, ST-Raptor, and the budget-matched Self-Consistency baseline by 13.62%, 11.71%, 9.34%, 4.39%, and 2.80%, respectively. With GPT-4o, TQA-AGENT achieves an average accuracy of 73.38% and outperforms the budget-matched Self-Consistency baseline by 2.70%. With Qwen2.5-7B-Instruct, TQA-AGENT achieves an average accuracy of 60.06%, outperforming Self-Consistency by 3.07 percentage points and ST-Raptor by 4.60 percentage points. These results confirm that the performance improvements are robust across backbone models.

To evaluate whether the observed improvements are robust rather than attributable to random variation, we repeat the DeepSeek-V3 experiments five times with different random seeds and report the mean accuracy and sample standard deviation in Table 3. We restrict the repeated-run significance analysis to DeepSeek-V3 to balance statistical rigor and computational cost. Specifically, we perform paired bootstrap tests with 10,000 resamples and McNemar tests comparing TQA-AGENT with ST-Raptor, the strongest original baseline. We additionally include a budget-matched Self-Consistency baseline, which generates multiple independent reasoning trajectories and selects the final answer via semantic majority voting under a comparable token budget. The significance tests confirm that the gains over ST-Raptor are robust across random runs, while the budget-matched comparison with Self-Consistency shows that the improvements cannot be explained by increased sampling alone.

Base Model Comparison. Methods equipped with GPT-4o generally achieve stronger performance, reflecting the benefit of a more capable backbone. Across different base models, TQA-AGENT consistently maintains a leading position with performance trends similar to other baselines. On average, TQA-AGENT improves from 71.56% on DeepSeek-V3 to 73.38% on GPT-4o, an absolute gain of 1.82%. This relatively modest improvement indicates that TQA-AGENT is not highly sensitive to the choice of base model, and its gains mainly stem from its internal mechanisms. At the dataset level, TQA-AGENT shows consistent improvements across models, e.g., from 73.20% to 76.20% on TReB and from 75.62% to 77.03% on SSTQA. These results suggest that while GPT-4o provides strong general capabilities (Achiam et al., 2023; Hurst et al., 2024), the structural parsing and dynamic search mechanisms of TQA-AGENT play a more critical role in its performance.

To further investigate backbone generalization, we evaluate TQA-AGENT with Qwen2.5-7B-Instruct, a smaller open-source instruction-tuned model. As expected, all methods experience performance degradation with the weaker backbone, reflecting the impact of model capacity on table perception, action planning, state evaluation, and trajectory selection. Despite this challenge, TQA-AGENT consistently outperforms all baselines across the four datasets, achieving an average accuracy of 60.06% and an average improvement of 3.07 percentage points over Self-Consistency and 4.60 percentage points over the strongest original baseline, ST-Raptor. These results demonstrate that the proposed structured search and trajectory-level deliberation mechanisms generalize beyond frontier-scale models and remain effective even under constrained backbone capabilities, while stronger backbones further enhance the quality of intermediate reasoning and decision-making.

Domain Comparison. TQA-AGENT demonstrates consistently strong results across datasets from financial (CRT, TReB), scientific (SciTab), and general (SSTQA) domains. In the financial domain, it achieves approximately 9% improvement on CRT and 3% on TReB compared with the strongest baselines. These datasets feature multi-level headers, cross-region merges, and structural jumps, which pose challenges for table reasoning. TQA-AGENT’s structural parsing and dynamic search mechanisms allow it to handle these challenges effectively, and the integration of FIBO as a domain-specific knowledge source further supports accurate cross-column referencing and metric understanding. In the scientific domain, TQA-AGENT maintains the highest accuracy, reaching 63.59% on DeepSeek-V3 and 65.27% on GPT-4o, even when the knowledge source is switched to ConceptNet-Science. This demonstrates that the method can adapt to domain-specific terminology and experimental metrics while preserving high accuracy. For the general domain SSTQA dataset, which contains highly heterogeneous tables, TQA-AGENT achieves 75.62% with DeepSeek-V3 and 77.03% with GPT-4o, outperforming all baseline methods. These results indicate that TQA-AGENT generalizes well across domains with varying linguistic and reasoning characteristics.

Structural Complexity Comparison. Beyond domain variation, TQA-AGENT exhibits stable performance on tables of varying structural complexity. CRT tables are relatively standardized financial statements with consistent layouts, while TReB tables are more

Table 4
Ablation study of reasoning actions in TQA-AGENT on CRT and TReB.

Actions	CRT	TReB
All actions (Ours)	73.83%	73.20%
- w/o tabular information retrieval actions	58.03% (↓ 15.80%)	52.80% (↓ 20.40%)
- w/o reasoning actions	64.01% (↓ 9.82%)	68.20% (↓ 5.00%)
- w/o computing action	54.09% (↓ 19.74%)	67.00% (↓ 6.20%)
- w/o external knowledge retrieval actions	68.04% (↓ 5.79%)	72.00% (↓ 1.20%)
- w/o multi-hop question decomposition actions	71.99% (↓ 1.84%)	71.60% (↓ 1.60%)

structurally diverse, including commercial analysis charts, cross-region layouts, and higher proportions of merged or hierarchical cells. Despite these differences, TQA-AGENT achieves consistently high accuracy on both datasets (e.g., 73.83% on CRT and 73.20% on TReB with DeepSeek-V3). In contrast, baseline methods such as Direct-Prompting and TableLLM show larger fluctuations in performance when table structures become more complex. This stability highlights that TQA-AGENT’s reasoning process, combining structural parsing, search-based reasoning chain construction, and knowledge completion, is well aligned with complex table layouts and robust against structural variability.

Cost Analysis. We analyze the computational cost of TQA-AGENT by measuring the average token consumption per question. Compared with single-pass baselines, TQA-AGENT incurs higher token usage. On DeepSeek-V3, it consumes 23.2k tokens per question on average, versus 1.8k, 2.2k, 4.2k, and 5.7k for Direct-Prompting, TableLLM, MACT, and ST-Raptor, respectively. A similar trend is observed with GPT-4o, where TQA-AGENT uses 25.5k tokens on average, compared with 1.9k–6.3k for the original baselines. This higher cost reflects the additional computation devoted to structured search and trajectory-level reasoning.

To enable a controlled comparison under comparable inference budgets, we further introduce a budget-matched Self-Consistency baseline. Self-Consistency consumes 23.1k, 25.3k, and 21.1k tokens with DeepSeek-V3, GPT-4o, and Qwen2.5-7B-Instruct, respectively, closely matching the corresponding computational budgets of TQA-AGENT. Although Self-Consistency improves over several lower-cost baselines, it consistently underperforms TQA-AGENT across all datasets and backbone models. On average, TQA-AGENT achieves gains of 2.80, 2.70, and 3.07 percentage points over Self-Consistency with DeepSeek-V3, GPT-4o, and Qwen2.5-7B-Instruct, respectively, under nearly identical token budgets.

These results indicate that the performance improvements of TQA-AGENT cannot be explained by token consumption alone. Repeated independent sampling improves answer-level diversity, but it does not maintain shared search statistics, perform value back-propagation over structured table states, or compare complete executable trajectories. In contrast, TQA-AGENT organizes the additional computation through structured state-action search and trajectory-level deliberation, which leads to more reliable reasoning than simply generating more independent answers.

Summary for RQ1. The experimental results demonstrate that TQA-AGENT consistently outperforms existing methods across diverse language models, application domains, and table structures. The improvements are mainly achieved through structured search and trajectory-level deliberation, which enable more reliable reasoning beyond the benefits of additional computational budget.

4.3. Ablation of reasoning actions (RQ2)

To investigate the contribution of each action category to TQA-AGENT, we perform an ablation study by systematically removing one action category at a time (excluding *Finish*) and re-evaluating the model. Experiments are conducted using DeepSeek-V3 on the CRT and TReB datasets. The results in Table 4 show that removing any single action category leads to a noticeable and consistent drop in performance across both datasets. This indicates that none of the action types is individually sufficient, and the overall effectiveness of TQA-AGENT relies on the coordinated interplay among multiple reasoning actions.

Table Information Retrieval Ablation. Excluding table information retrieval actions produces the most substantial performance drop. Accuracy on CRT decreases from 73.83% to 58.03%, and on TReB from 73.20% to 52.80%. These actions are responsible for fundamental table understanding, including multi-level header expansion, column attribute localization, and target row selection, forming the starting point for subsequent reasoning and computation. Without precise localization of relevant table regions, downstream reasoning cannot reliably build on structured inputs. The performance decline is particularly pronounced on the more heterogeneous TReB dataset, where structural variability amplifies the impact of missing retrieval capabilities.

Computing Ablation. Removing computing actions also results in significant performance degradation, particularly on CRT, where accuracy drops from 73.83% to 54.09%, while TReB sees a smaller decline from 73.20% to 67.00%. Computing actions enable precise numerical operations over typical financial metrics such as year-over-year or month-over-month changes, ratios, and totals. Without these explicit arithmetic capabilities, questions requiring exact calculations cannot be consistently answered, especially in CRT where such queries are frequent. In TReB, more questions involve interpretive or explanatory reasoning, while fewer depend on strict calculations, resulting in a smaller but still noticeable impact.

Reasoning Ablation. Excluding reasoning actions causes a clear performance drop, with accuracy falling from 73.83% to 64.01% on CRT and from 73.20% to 68.20% on TReB. Reasoning actions support the construction of intermediate representations, such as derived rows or columns, and facilitate multi-step logical operations including sorting, grouping, and comparison. Their effect

Table 5

Ablation results of key components in TQA-AGENT on CRT and TReB.

Method	CRT		TReB	
	Accuracy	Avg. tokens	Accuracy	Avg. tokens
TQA-AGENT	73.83% $\pm$ 0.31	21.7k	73.20% $\pm$ 0.34	24.5k
TQA-AGENT (Independent Sampling)	70.18% $\pm$ 1.91	25.8k	70.60% $\pm$ 1.72	28.4k
TQA-AGENT (Single Trajectory)	70.25%	5.2k	71.80%	6.5k
TQA-AGENT (Majority Voting)	71.64%	20.9k	71.60%	23.7k
TQA-AGENT (Single Agent)	68.34%	19.3k	67.40%	24.1k

is particularly visible in the more structured CRT dataset, where many questions require stepwise computation and comparative analysis. In TReB, a portion of questions can be answered through direct retrieval or simple computation, so the impact is slightly smaller but still meaningful.

External Knowledge Retrieval Ablation. Ablating external knowledge retrieval actions causes a relatively moderate performance drop, from 73.83% to 68.04% on CRT and from 73.20% to 72.00% on TReB. These actions primarily support the interpretation of domain-specific terminology and implicit semantic relations beyond the table contents. Since many questions can be answered using information contained within the table itself, external knowledge mainly benefits specialized or semantically nuanced cases. To further evaluate retrieval quality, we manually analyze 40 randomly sampled cases involving external knowledge retrieval. Among them, 82.5% retrieve correct and useful domain knowledge that directly supports reasoning, while 12.5% fail due to knowledge base coverage limitations and 5.0% return incorrect or misleading information. These results indicate that the retrieval mechanism is reliable in most cases and that the underlying financial knowledge sources provide reasonable coverage of the concepts required by the benchmarks, consistent with the moderate yet consistent gains observed.

Multi-hop Question Splitting Ablation. Removing multi-hop question splitting actions results in the smallest performance decrease: from 73.83% to 71.99% on CRT and from 73.20% to 71.60% on TReB. These actions target complex questions requiring serial or parallel reasoning by decomposing them into manageable sub-questions. While such questions are less prevalent in financial datasets, this capability still contributes positively to handling more intricate reasoning scenarios.

Summary for RQ2. The ablation results demonstrate that the proposed action space is both effective and complementary, with each action category making a distinct contribution to overall performance. Table retrieval, computation, and reasoning actions provide the foundation for structural understanding and numerical reasoning, while external knowledge retrieval and multi-hop question splitting further enhance semantic understanding and complex problem solving.

4.4. Ablation of key components (RQ3)

To quantify the contribution of each key component in TQA-AGENT, we conduct systematic ablation studies on the CRT and TReB benchmarks using DeepSeek-V3. We successively remove (1) tree-search-guided reasoning, (2) trajectory-level deliberation, (3) deliberation-based trajectory selection, and (4) role-specialized collaboration. Table 5 reports the resulting answer accuracy, token consumption, and, where applicable, standard deviations over repeated stochastic executions.

Effect of Tree Search. A central question is whether the gains of TQA-AGENT result from structured tree-search exploration or simply from increased sampling. To disentangle these effects, we compare TQA-AGENT with an independent sampling baseline. Specifically, each question is solved by five independent reasoning runs, where each run generates a complete reasoning trajectory without shared search state or tree reuse and terminates upon the first Finish action. The final prediction is obtained by majority voting over the five independently generated answers.

The independent sampling baseline achieves mean accuracies of 70.18% (± 1.91) on CRT and 70.60% (± 1.72) on TReB, whereas TQA-AGENT reaches 73.83% (± 0.31) and 73.20% (± 0.34), respectively. These improvements indicate that the gains of TQA-AGENT cannot be explained solely by increased stochastic sampling. Instead, they demonstrate the benefit of tree-search-guided exploration, which refines promising trajectories through shared search statistics and value-informed search decisions.

Beyond accuracy improvements, tree-search-guided exploration also enhances reasoning stability. Compared with the independent sampling baseline, TQA-AGENT achieves substantially lower standard deviations (0.31 vs. 1.91 on CRT and 0.34 vs. 1.72 on TReB), indicating that shared search statistics reduce the variance introduced by independent stochastic generation. These results demonstrate that structured tree search improves both predictive performance and inference stability across repeated runs.

Effect of Multi-Agent Deliberation. To examine the effect of multi-agent trajectory deliberation, we compare TQA-AGENT ($K = 5$) with a variant that sets the candidate budget to $K = 1$, thereby removing cross-trajectory evaluation and trajectory selection. In this setting, the final prediction is determined solely by the single trajectory generated by the tree-search procedure, allowing us to isolate the contribution of deliberation over multiple candidate trajectories.

As shown in Table 5, introducing trajectory-level deliberation consistently improves performance across both benchmarks, yielding accuracy gains of 3.58 percentage points on CRT and 1.40 percentage points on TReB. These results indicate that relying on a single reasoning trajectory is insufficient to fully exploit the search space, even with tree-search-guided exploration. By jointly examining multiple high-quality candidate trajectories, agents can compare, critique, and reconcile alternative reasoning paths

Table 6
Search hyperparameter sensitivity of TQA-AGENT on CRT and TReB with DeepSeek-V3.

Top- m actions	K (paths)	Consistency weights	CRT		TReB	
			Accuracy	Avg. tokens	Accuracy	Avg. tokens
5	3	$\lambda_1 = 0.6, \lambda_2 = 0.4$	72.54%	13.6k	72.20%	15.4k
5	5	$\lambda_1 = 0.6, \lambda_2 = 0.4$	73.83%	21.7k	73.20%	24.5k
5	7	$\lambda_1 = 0.6, \lambda_2 = 0.4$	74.10%	29.8k	73.20%	31.8k
3	5	$\lambda_1 = 0.6, \lambda_2 = 0.4$	72.96%	20.5k	72.60%	23.1k
7	5	$\lambda_1 = 0.6, \lambda_2 = 0.4$	73.55%	22.6k	72.80%	25.3k
5	5	$\lambda_1 = 0.5, \lambda_2 = 0.5$	73.70%	21.7k	73.00%	24.5k
5	5	$\lambda_1 = 0.7, \lambda_2 = 0.3$	73.48%	21.7k	72.80%	24.5k

before reaching a final decision. This collaborative process helps identify reasoning inconsistencies that may remain undetected in single-path inference, thereby improving prediction quality beyond search-based exploration alone.

Effect of Trajectory-level Deliberation-based Selection. To isolate the contribution of deliberation-based trajectory selection, we compare TQA-AGENT with a variant that selects the final prediction through majority voting. Both variants use the same tree-search procedure and candidate budget ($K = 5$), differing only in how candidate trajectories are aggregated for final prediction.

Under this setting, majority voting achieves accuracies of 71.64% on CRT and 71.60% on TReB, whereas the deliberation-based selection mechanism reaches 73.83% and 73.20%, yielding improvements of 2.19 and 1.60 percentage points, respectively. As both variants use identical candidate trajectories, the performance gains are attributed to the improved trajectory selection strategy, with trajectory generation held constant.

The results suggest that simply aggregating answers is less effective than deliberating over complete reasoning trajectories. Majority voting selects the most frequent final answer without considering the quality or consistency of the underlying reasoning, whereas deliberation-based selection explicitly evaluates competing reasoning paths and their supporting evidence before reaching a consensus. These results demonstrate that reasoning-aware trajectory evaluation improves prediction reliability beyond simple aggregation based on final-answer frequency.

Effect of Role-specialized Collaboration. To assess the effect of role-specialized collaboration, we compare TQA-AGENT with a single-agent baseline that preserves the same action space, tree-search procedure, maximum search depth, candidate path budget, and deterministic executor. The only difference is that a single LLM performs all reasoning functions, including state perception, action planning, state evaluation, termination judgment, and final trajectory selection, instead of distributing these responsibilities across specialized agents.

The single-agent baseline achieves accuracies of only 68.34% on CRT and 67.40% on TReB, substantially lower than the 73.83% and 73.20% obtained by TQA-AGENT. The corresponding performance gains of 5.49 and 5.80 percentage points indicate that the improvements of TQA-AGENT cannot be attributed solely to tree-search exploration, the action space, or repeated execution. Instead, they highlight the importance of role specialization within the multi-agent architecture.

A single agent is required to handle multiple reasoning functions with different objectives, which may lead to conflicts among these processes. By assigning dedicated responsibilities to specialized agents, TQA-AGENT enables more reliable state evaluation, better-informed action planning, and more effective trajectory selection. These findings suggest that role-specialized collaboration is an essential component of TQA-AGENT, complementing tree-search exploration with more effective coordination among different reasoning functions.

Summary for RQ3. The ablation results confirm that the performance gains of TQA-AGENT come from the complementary contributions of its key components. Tree-search exploration improves trajectory discovery, trajectory-level deliberation strengthens cross-trajectory evaluation, deliberation-based selection achieves more reliable final decisions than simple voting, and role specialization facilitates effective coordination among reasoning functions.

4.5. Impact of hyperparameters (RQ4)

We investigate the sensitivity of TQA-AGENT to search-specific hyperparameters that affect different stages of the reasoning process, including the number of candidate actions retained during action selection (top- m), the number of completed reasoning trajectories collected before terminating tree search (K), and the weighting coefficients for the cross-trajectory consistency score. We further analyze reward-related configurations in the state evaluation function, including progress-feature weights, path-length penalties, and terminal reward values. All experiments are conducted with DeepSeek-V3 on CRT and TReB while keeping other configurations unchanged. We report both answer accuracy and average token consumption to characterize the trade-off between reasoning effectiveness and computational cost. Unless otherwise specified, the default configuration is top- $m = 5$, $K = 5$, and $(\lambda_1, \lambda_2) = (0.6, 0.4)$.

Search Parameter Sensitivity. Table 6 shows that different search parameters affect TQA-AGENT in distinct ways. The trajectory exploration parameters (K and top- m) primarily determine the exploration–efficiency trade-off, while the cross-trajectory consistency scoring weights mainly influence final trajectory ranking and have a relatively limited impact on overall accuracy. Specifically,

Table 7

Reward configuration sensitivity of TQA-AGENT on CRT and TReB with DeepSeek-V3.

Reward setting	CRT		TReB	
	Accuracy	Avg. tokens	Accuracy	Avg. tokens
Default	73.83%	21.7k	73.20%	24.5k
Uniform progress weights	73.34%	21.9k	72.80%	24.8k
No depth penalty	73.02%	23.1k	72.60%	26.2k
Lower terminal reward	73.21%	21.4k	72.80%	24.1k
Higher terminal reward	73.65%	21.8k	73.00%	24.6k
Stronger missing-answer penalty	73.45%	21.6k	72.60%	24.3k

Table 8

Human evaluation results. The results show strong agreement between human evaluation and the LLM-based judge across all datasets.

Dataset	Human Evaluation Acc.	LLM Judge Acc.	Human-Judge Agreement	Cohen's Kappa
CRT	68.0%	68.0%	96.0%	0.91
TReB	66.0%	66.0%	92.0%	0.82
SciTab	60.0%	62.0%	94.0%	0.87
SSTQA	74.0%	70.0%	96.0%	0.90
Overall	67.0%	66.5%	94.5%	0.88

K determines the number of candidate trajectories explored during tree search. Increasing K from 3 to 5 consistently improves accuracy on both datasets, indicating that additional trajectory exploration provides more informative evidence for final selection. However, further increasing K to 7 increases token consumption by 37.3% on CRT and 29.8% on TReB, while bringing only marginal improvement on CRT and no gain on TReB. This suggests that excessive exploration introduces diminishing returns. Similarly, top- m determines the branching factor of the search tree by controlling the number of candidate actions considered at each decision step. A small value ($m = 3$) limits exploration and reduces reasoning accuracy, whereas a larger value ($m = 7$) substantially expands the search space with limited performance benefits. The default setting ($m = 5$) therefore achieves a favorable balance between exploration coverage and computational cost. In comparison, varying the cross-trajectory consistency scoring weights leads to only minor performance fluctuations. Both alternative configurations, (0.5, 0.5) and (0.7, 0.3), achieve comparable results to the default setting but do not consistently outperform (0.6, 0.4). This indicates that self-evaluation and cross-trajectory consistency provide complementary signals, and that their balanced integration enables more robust trajectory ranking than relying heavily on either signal.

Reward Configuration Sensitivity. We further examine the robustness of TQA-AGENT to variations in the reward configuration used by the state evaluation function. Specifically, we perturb the non-terminal progress weights, remove the path-length penalty, and vary terminal reward values while keeping all other search parameters fixed. As shown in Table 7, these reward variations lead to only minor accuracy changes, generally within one percentage point. Removing the path-length penalty slightly increases token consumption and decreases accuracy, indicating that length regularization helps avoid unnecessary reasoning steps during search. Overall, the default reward configuration achieves a stable balance among search progress, answer completion, and computational efficiency, suggesting that TQA-AGENT is robust to reasonable reward variations.

Summary for RQ4. TQA-AGENT exhibits stable performance under moderate hyperparameter variations. The trajectory exploration parameters (K and top- m) mainly control the exploration-cost trade-off, while the cross-trajectory consistency scoring weights and reward configuration have limited influence on final accuracy. The default setting (top- $m = 5$, $K = 5$, and $(\lambda_1, \lambda_2) = (0.6, 0.4)$) provides an effective balance between reasoning performance and computational cost.

4.6. Human evaluation (RQ5)

While the LLM-based judge offers a scalable and cost-effective evaluation protocol, its semantic judgments may still be affected by potential biases. To assess its reliability, we conduct an additional human evaluation study. Specifically, we randomly sample 200 prediction instances using stratified sampling, with 50 instances from each dataset. Two professional engineers independently determine whether each prediction is semantically consistent with the gold answer based on the original table-question instance, the gold answer, and the model prediction. For CRT and TReB, we retain the original Chinese-source financial table content to ensure faithful evaluation. Disagreements are resolved through adjudication to obtain the final human labels.

The results are presented in Table 8. Human evaluation yields an overall accuracy of 67.0%, while the LLM-based judge yields 66.5%; their overall agreement is 94.5% with a Cohen's Kappa score of 0.88, indicating strong consistency with human judgments. At the dataset level, the agreement rates remain consistently high, ranging from 92.0% to 96.0%, demonstrating the robustness of the judge across different table QA scenarios. The remaining disagreements mainly arise from numerical formatting or rounding differences, semantically equivalent expressions, ambiguous partial answers, or boundary cases involving multiple acceptable responses rather than systematic failures in cross-lingual semantic evaluation.

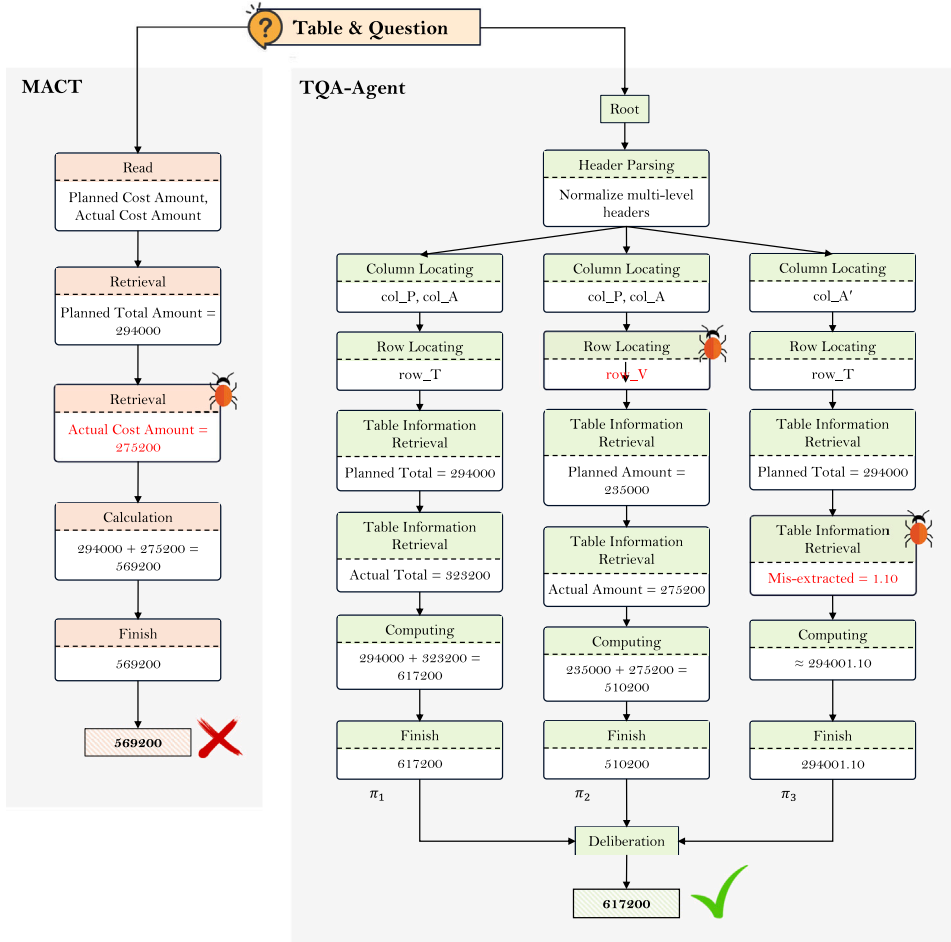


Fig. 4. Reasoning case study of TQA-AGENT vs. MACT. MACT (left) follows a single linear reasoning path and mistakenly extracts the *Actual Total Amount* from a sub-row, leading to an incorrect final result. In contrast, TQA-AGENT (right) explores three candidate reasoning paths, two of which fail due to column misalignment and row mislocalization, respectively. The remaining path correctly aggregates the *Planned* and *Actual Total Amounts*, and is selected through trajectory-level deliberation to produce the final correct answer.

Furthermore, the judge maintains comparable reliability on the Chinese-source financial datasets. Specifically, CRT achieves 68.0% human accuracy and 68.0% LLM-judge accuracy, with 96.0% agreement and a Cohen's Kappa of 0.91, while TReB achieves 66.0% human accuracy and 66.0% LLM-judge accuracy, with 92.0% agreement and a Cohen's Kappa of 0.82. These results are comparable to those on SciTab and SSTQA, suggesting that the LLM-based judge does not suffer from degraded reliability when evaluating cross-lingual financial table QA.

Summary for RQ5. Our human evaluation demonstrates that the LLM-based judge closely aligns with human judgments, achieving high agreement across all evaluated datasets. Its consistent performance on Chinese-source financial tables further verifies its reliability for semi-structured table QA evaluation.

5. Case study

We use the motivating example in Fig. 1, which asks for the total planned and actual unit costs, to illustrate the differences between MACT and TQA-AGENT. MACT formulates the task as a single-path sequential reasoning process, committing to each intermediate decision without revision. In this case, although MACT correctly identifies the Planned Total Amount (294 000), it misaligns the table structure when extracting the Actual Amount, erroneously selecting the value from the Variable Cost row (275 200). This early structural error cannot be corrected and consequently propagates to the final prediction, resulting in an incorrect total of 569 200.

In contrast, TQA-AGENT performs parallel exploration over a tree-structured space of reasoning paths, where different branches correspond to alternative alignments of hierarchical headers, row selections, and interpretations of the Total semantics. As

shown in Fig. 4, some candidate reasoning paths are invalid: one aligns the Actual Amount with the Completion Percentage column, yielding a unit-inconsistent sum of 294 001.10, while another misidentifies the Total row and produces 510 200. Through trajectory-level deliberation, these flawed paths are explicitly evaluated and discarded based on inconsistencies in their intermediate reasoning. The Decision Agent ultimately selects the correct path, which aggregates the Planned and Actual Total Amounts (294 000 and 323 200) to produce the correct answer of 617 200. This case study demonstrates that multi-path exploration with comparative evaluation effectively mitigates error propagation in semi-structured Table QA, whereas single-path methods such as MACT remain vulnerable to early missteps.

6. Discussion

6.1. Why is TQA-AGENT effective?

We believe that the effectiveness of TQA-AGENT primarily arises from a fundamental shift in how semi-structured Table QA is formulated: from single-pass heuristic reasoning to structured, stateful search over executable operations. Rather than committing early to a fixed interpretation of table structure or a single reasoning trajectory, TQA-AGENT models reasoning as a sequential decision process within an explicit action space. This reformulation directly addresses three key challenges in semi-structured tables: structural ambiguity, combinatorial search complexity, and strong path dependency.

First, structured state-action modeling reduces semantic drift. In conventional LLM-based approaches, tables are linearized into text and processed through unconstrained generation, where structural constraints are only implicitly represented. As a result, hierarchical relations, alignment dependencies, and aggregation scopes can be easily distorted. By representing reasoning as executable table operations over explicit intermediate states, TQA-AGENT preserves structural consistency throughout the reasoning process. Moreover, well-defined state transitions allow the tree-search procedure to maintain reliable search statistics, resulting in more stable exploration compared with unconstrained natural-language reasoning.

Second, structured search mitigates error accumulation in multi-step reasoning. Unlike existing MCTS-based LLM methods that search over textual thoughts or reasoning traces, TQA-AGENT performs exploration directly over executable table states. This state-based search enables the evaluation of alternative structural decisions, including table interpretation, row and column selection, aggregation scope, and operation ordering. By maintaining multiple candidate trajectories, TQA-AGENT supports continuous comparison and refinement of competing hypotheses, while preventing premature commitment to potentially erroneous decisions.

Third, the trajectory-level deliberation mechanism enhances robustness through trajectory-level verification. The specialized agents assess complete executable trajectories by examining intermediate table states, structural decisions, and final answers jointly. This trajectory-level analysis reveals grounding inconsistencies that may remain hidden when only final answers are considered, providing a more informative verification signal for final trajectory selection.

More broadly, TQA-AGENT demonstrates how symbolic structure and neural reasoning can be integrated within a unified inference framework. Explicit state transitions provide interpretability and constraint awareness, while LLM agents contribute flexible semantic understanding and domain adaptability. Their synergy explains the consistent performance gains observed across diverse domains and semi-structured table formats.

6.2. Limitations and future directions

Despite the promising results, our work has several limitations that motivate future research.

First, TQA-AGENT is designed as a task-specific framework that integrates structured action spaces, search-based reasoning, and multi-agent trajectory verification for semi-structured Table QA. It is not intended to introduce a general-purpose search algorithm or a new theoretical reasoning paradigm. Instead, its contribution lies in operationalizing these components into an executable state-action reasoning process and demonstrating through controlled experiments that the performance gains arise from structured reasoning organization rather than increased computation alone.

One limitation is that TQA-AGENT relies on a predefined table-centric action space and external knowledge retrieval. Although the action space provides an effective interface between LLMs and semi-structured tables, it may not cover all reasoning behaviors required in more diverse or unseen environments. Meanwhile, inaccurate or incomplete retrieved knowledge can still affect downstream reasoning, despite retrieval-related errors accounting for only a limited proportion of failures in our analysis. Future work will explore adaptive action-space construction, learned action discovery, retrieval verification, uncertainty-aware retrieval, and self-correction mechanisms to improve flexibility and robustness.

Another limitation concerns the generalizability of our evaluation. Although experiments with Qwen2.5-7B-Instruct demonstrate that TQA-AGENT remains effective beyond frontier-scale models, the current evaluation still covers a limited range of model families and data scenarios. Future work will extend validation to broader model architectures and scales, as well as more diverse structured data settings, such as hybrid table-text reasoning. In addition, while we analyze the sensitivity of key framework hyperparameters, a more systematic study of the remaining tree-search configurations could provide deeper insights into search behavior and trajectory optimization.

Finally, TQA-AGENT introduces additional inference overhead due to multi-path exploration and trajectory-level deliberation compared with single-path prompting. This reflects a trade-off between computational cost and reasoning reliability. Our cost analysis shows that the performance improvements are not merely a result of increased token consumption, but arise from more effective search and trajectory-level verification. Future work will investigate lightweight agentic reasoning strategies, including adaptive budget allocation, effective path pruning, early stopping, and learned value estimation, to reduce search overhead while preserving reasoning quality.

6.3. Implications

Theoretical Implications. This work suggests a conceptual shift in modeling LLM-based reasoning over structured data, moving from end-to-end text generation toward explicit stateful decision processes based on executable operations. By introducing an action space and formal state transitions, TQA-AGENT provides a concrete framework for integrating symbolic structural constraints with neural semantic reasoning within a unified inference process. Furthermore, the findings suggest that reliable reasoning in complex tasks can benefit from structured exploration and evaluation of multiple candidate trajectories. Combining controlled search with trajectory-level verification provides a principled approach for handling uncertainty and ambiguity in structured data reasoning. The agent-based functional decomposition further offers a reusable abstraction for organizing complex reasoning systems, suggesting that future LLM architectures may benefit from explicit functional role separation and coordination beyond monolithic prompting strategies.

Practical Implications. From a practical standpoint, the framework offers a blueprint for building trustworthy table reasoning systems in high-stakes domains where interpretability and correctness are essential. Explicit intermediate states and complete reasoning paths naturally support auditing, debugging, and post-hoc verification. The modular architecture facilitates domain adaptation: new operations, knowledge sources, or reasoning constraints can be incorporated without redesigning the entire system. Moreover, the framework enables a controllable reliability–efficiency trade-off. By adjusting search depth, candidate trajectory count, or deliberation budget, system designers can tailor inference behavior to available computational budgets and deployment requirements.

7. Conclusion

In this paper, we introduce TQA-AGENT, a multi-agent reasoning framework that reformulates semi-structured Table QA as a structured search problem over an explicit space of executable semantic actions. To effectively reason over complex semi-structured tables, TQA-AGENT leverages tree-search-guided exploration to construct and refine multiple trajectories, while a trajectory-level deliberation mechanism adjudicates among competing paths to identify the most reliable answer. Extensive experiments on four benchmarks — CRT, TREB, SciTab, and SSTQA — demonstrate that TQA-AGENT consistently surpasses state-of-the-art LLM-based approaches, notably achieving 73.20% accuracy on the challenging TREB dataset.

CRedit authorship contribution statement

Jianghong Huang: Writing – original draft, Visualization, Validation, Project administration, Methodology. **Shangbo Yun:** Writing – review & editing, Investigation, Formal analysis, Data curation, Conceptualization. **Silin Chen:** Writing – review & editing, Supervision. **Xiaodong Gu:** Writing – review & editing, Supervision. **Beijun Shen:** Writing – review & editing, Supervision, Software, Resources, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research is funded by the National Natural Science Foundation of China (Grant No. 62232003), and the Natural Science Foundation of Shanghai (Grant No. 25ZR1401175).

Data availability

The data and code supporting the findings of this study are publicly available at the GitHub repository provided in the manuscript. [artifact-PoA \(Original data\)](#) (Github)

References

- Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altschmidt, J., Altman, S., Anadkat, S., et al. (2023). GPT-4 technical report. arXiv preprint [arXiv:2303.08774](https://arxiv.org/abs/2303.08774).
- Agarwal, A., Devaguptapu, C., & Ganesh, S. (2025). Hybrid graphs for table-and-text based question answering using LLMs. In *Proceedings of NAACL 2025* (pp. 858–875). [http://dx.doi.org/10.18653/v1/2025.naacl-long.39](https://doi.org/10.18653/v1/2025.naacl-long.39).
- Bansal, H., Raj, A., Sharma, A., & Krishnamurthy, P. (2025). TableWise at SemEval-2025 task 8: LLM agents for tabular QA. In *Proceedings of SemEval-2025*.
- Browne, C. B., Powley, E., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., & Colton, S. (2012). A survey of monte carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 1–43.
- Chen, W., Zha, H., Chen, Z., Wang, W., & Wang, W. Y. (2021). HybridQA: A dataset of multi-hop question answering over tabular and textual data. In *Proceedings of the 59th annual meeting of the association for computational linguistics* (pp. 1026–1036). [http://dx.doi.org/10.18653/v1/2021.acl-long.43](https://doi.org/10.18653/v1/2021.acl-long.43).

- Chen, W., Zha, H., Chen, Z., Xiong, W., Wang, H., & Wang, W. Y. (2020). HybridQA: A dataset of multi-hop question answering over tabular and textual data. In *Findings of the association for computational linguistics: EMNLP 2020* (pp. 1026–1036).
- DeepSeek-AI (2024). DeepSeek-V3 technical report. arXiv:2412.19437, URL: <https://arxiv.org/abs/2412.19437>.
- Gupta, N., Mehta, S., Singh, S., & Roth, D. (2020). INFOTABS: Inference on tables as semi-structured data. In *Proceedings of the AAAI conference on artificial intelligence*.
- Herzig, J., Nowak, P. K., Müller, T., Piccinno, F., & Eisenschlos, J. (2020). TaPas: Weakly supervised table parsing via pre-training. In *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 4320–4333).
- Hurst, A., Lerer, A., Goucher, A. P., Perelman, A., Ramesh, A., Clark, A., Ostrow, A., Welihinda, A., Hayes, A., Radford, A., et al. (2024). GPT-4o system card. arXiv preprint arXiv:2410.21276.
- Jiang, C., Cheng, M., Tao, X., Mao, Q., Ouyang, J., & Liu, Q. (2025). TableMind: An autonomous programmatic agent for tool-augmented table reasoning. arXiv preprint arXiv:2509.06278, URL: <https://arxiv.org/abs/2509.06278>, Preprint.
- Jiang, Z., Mao, Y., He, P., Neubig, G., & Chen, W. (2022). OmniTab: Pretraining with natural and synthetic data for few-shot table-based question answering. In *Proceedings of the 2022 conference of the North American chapter of the association for computational linguistics: Human language technologies* (pp. 932–942). Seattle, United States: Association for Computational Linguistics.
- Jiang, J., Zhou, K., Dong, Z., Ye, K., Zhao, X., & Wen, J.-R. (2023). StructGPT: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 9237–9251). Singapore: Association for Computational Linguistics.
- Li, C., Liu, X., Song, Z., Chi, C., Zhao, C., Yang, J., Wang, Z., Yang, K., Shi, B., Wang, X., Deng, C., & Feng, J. (2025). TRB: A comprehensive benchmark for evaluating table reasoning capabilities of large language models. ArXiv abs/2506.18421.
- Liu, Q., Chen, B., Guo, J., Ziyadi, M., Lin, Z., Chen, W., & Lou, J.-G. (2022). TAPEX: Table pre-training via learning a neural SQL executor. In *International conference on learning representations*.
- Liu, X., Li, J., Li, Z., et al. (2022). StructGPT: A general framework for large language model to reason over structured data. In *Proceedings of the 2022 conference on empirical methods in natural language processing* (pp. 9237–9251). <http://dx.doi.org/10.18653/v1/2022.emnlp-main.574>.
- Lu, X., Pan, L., Liu, Q., Nakov, P., & Kan, M.-Y. (2023). SCITAB: A challenging benchmark for compositional reasoning and claim verification on scientific tables. In *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 7787–7813). Singapore: Association for Computational Linguistics.
- Lu, W., Zhang, J., Fan, J., Fu, Z., Chen, Y., & Du, X. (2025). Large language model for table processing: A survey. *Frontiers of Computer Science*, 19(2), Article 192350.
- Nan, L., Hsieh, C., Mao, Z., Lin, X. V., Verma, N., Zhang, R., Kryściński, W., Schoelkopf, H., Kong, R., Tang, X., et al. (2022). FeTaQA: Free-form table question answering. *Transactions of the Association for Computational Linguistics*, 10, 35–49.
- Oguz, B., Lin, X. V., Yu, T., et al. (2022). UnifiedSKG: Unifying and multi-tasking structured knowledge grounding with text-to-text language models. In *Proceedings of the 2022 conference on empirical methods in natural language processing* (pp. 602–631). <http://dx.doi.org/10.18653/v1/2022.emnlp-main.39>.
- OpenAI (2024). GPT-4o system card. URL: <https://openai.com/index/gpt-4o-system-card/>, Available from OpenAI official documentation, Technical description of GPT-4o by OpenAI.
- Pan, F., Caim, M., Glass, M., Gliozzo, A., & Fox, P. (2021). CLTR: An end-to-end, transformer-based system for cell-level table retrieval and table question answering. In *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing: System demonstrations* (pp. 202–209). Online: Association for Computational Linguistics.
- Pasupat, P., & Liang, P. (2015). Compositional semantic parsing on semi-structured tables. In *Proceedings of the annual meeting of the association for computational linguistics*.
- Sholehrasa, H., Norouzi, S. S., Hitzler, P., & Jaber-Douraki, M. (2024). Knowledge in triples for LLMs: Enhancing table QA accuracy with semantic extraction. arXiv preprint arXiv:2409.14192.
- Tang, Z., Niu, B., Zhou, X., Li, B., Zhou, W., Wang, J., Li, G., Zhang, X., & Wu, F. (2025). St-Raptor: LLM-powered semi-structured table question answering. *Proceedings of the ACM on Management of Data*, 3(6), 1–27.
- Tyagi, R., Gupta, M., & Bouri, R. (2025). Aestar at SemEval-2025 task 8: Agentic LLMs for question answering over tabular data. In *Proceedings of semEval-2025*.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., & Cao, Y. (2022). React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.
- Yin, P., Neubig, G., Yih, W.-t., & Riedel, S. (2020). TaBERT: Pretraining for joint understanding of textual and tabular data. In *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 8413–8426). Online: Association for Computational Linguistics.
- Zhang, Y., Henkel, J., Floratou, A., Cahoon, J., Deep, S., & Patel, J. M. (2024). ReAcTable: Enhancing ReAct for table question answering. *Proceedings of the VLDB Endowment*, 17(8), 1981–1994.
- Zhang, Z., Li, X., Gao, Y., & Lou, J.-G. (2023). CRT-QA: A dataset of complex reasoning question answering over tabular data. In *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 2131–2153).
- Zhang, X., Luo, S., Zhang, B., Ma, Z., Zhang, J., Li, Y., Li, G., Yao, Z., Xu, K., Zhou, J., et al. (2025). TableLLM: Enabling tabular data manipulation by LLMs in real office usage scenarios. In *Findings of the association for computational linguistics: ACL 2025* (pp. 10315–10344).
- Zhang, Y., Maekawa, S., & Bhutani, N. (2025). Same content, different representations: A controlled study for table QA. arXiv preprint arXiv:2509.22983.
- Zhang, X., Shou, L., Pei, J., Gong, M., Wen, L., & Jiang, D. (2020). A graph representation of semi-structured data for web question answering. In *Proceedings of COLING 2020*.
- Zhang, T., Yue, X., Li, Y., & Sun, H. (2024). TableLlama: Towards open large generalist models for tables. In *Proceedings of the 2024 conference of the North American chapter of the association for computational linguistics: Human language technologies (volume 1: long papers)* (pp. 6024–6044).
- Zhou, W., Mesgar, M., Adel, H., & Friedrich, A. (2024). FREB-TQA: A fine-grained robustness evaluation benchmark for table question answering. In *Proceedings of the 2024 conference of the North American chapter of the association for computational linguistics: Human language technologies (volume 1: long papers)* (pp. 2479–2497). Mexico City, Mexico: Association for Computational Linguistics.
- Zhou, W., Mesgar, M., Friedrich, A., & Adel, H. (2025). Efficient multi-agent collaboration with tool use for online planning in complex table question answering. In *Findings of the association for computational linguistics: NAACL 2025* (pp. 945–968).
- Zhu, F., Lei, W., Huang, Y., Wang, C., Zhang, S., Lv, J., Feng, F., & Chua, T.-S. (2021). TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of ACL 2021* (pp. 3277–3287). Association for Computational Linguistics.